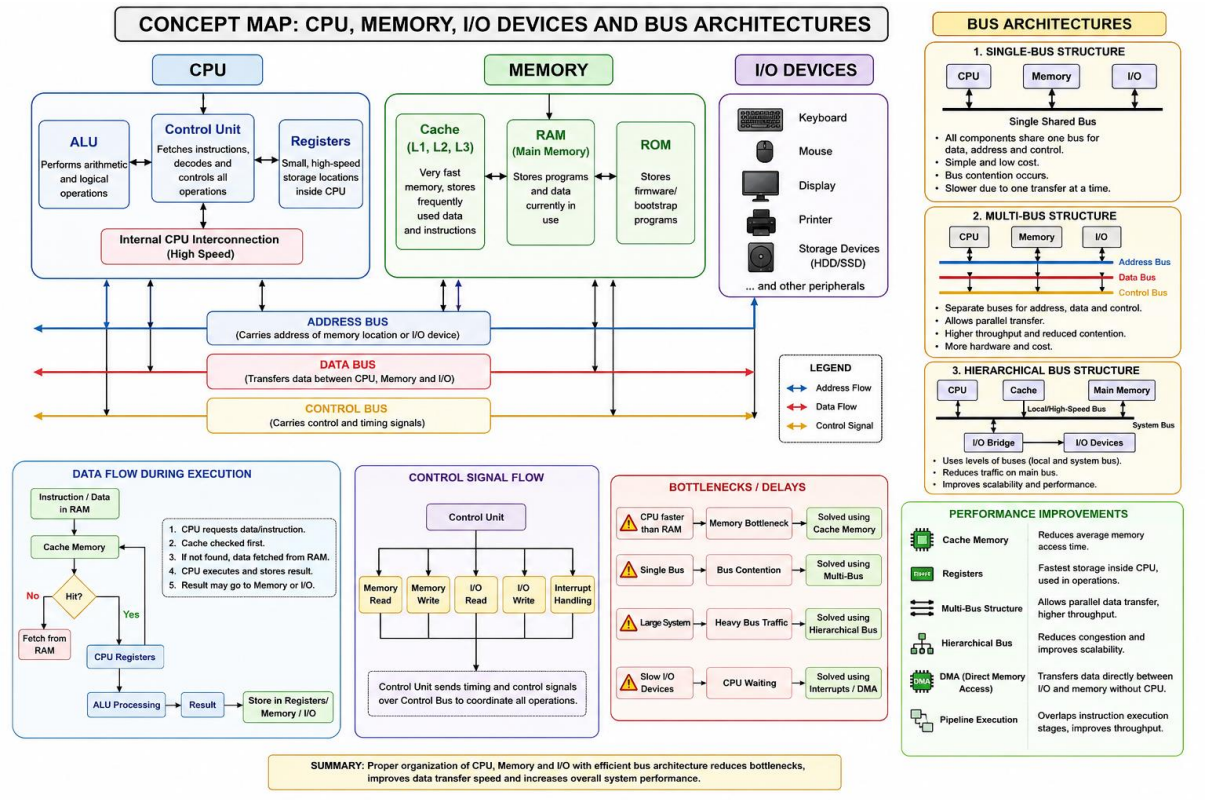
SIMATS ENGINEERING

ASSESSMENT TOOL CO2AT2

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA1221

Name: Ganesh Natarajan Reg.no:192412332

1.Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.



1. CPU (Central Processing Unit)

- The CPU is the central processing component responsible for executing program instructions and controlling overall system operations.
- It consists of three main units: the **Arithmetic Logic Unit (ALU)**, **Control Unit (CU)**, and **Registers**.

- The ALU performs arithmetic and logical operations, the Control Unit manages instruction execution, and the Registers provide high-speed temporary storage for data and instructions.

2. Memory

- The memory section includes **Cache Memory**, **RAM**, and **ROM**, each serving a specific purpose in data storage.
- Cache Memory stores frequently accessed data to reduce memory access time, while RAM temporarily stores active programs and data.
- ROM contains permanent firmware and boot programs required for system startup.

3. I/O Devices

- The concept map includes common input and output devices such as the keyboard, mouse, display, printer, and storage devices.
- These peripherals enable communication between the user and the computer system.
- Data is exchanged between the CPU and I/O devices through the system buses.

4. System Buses

- The **Address Bus** carries memory and I/O addresses from the CPU to other components.
- The **Data Bus** transfers data between the CPU, memory, and I/O devices during execution.
- The **Control Bus** carries timing and control signals, including memory read/write and interrupt signals, to coordinate system operations.

5. Bus Architectures

- The image compares **Single-Bus**, **Multi-Bus**, and **Hierarchical Bus** structures used in computer systems.
- A Single-Bus structure is simple but suffers from bus contention, whereas a Multi-Bus structure enables parallel communication for improved performance.

- A Hierarchical Bus structure reduces bus congestion by organizing communication into multiple levels, improving scalability and efficiency.

6. Data Flow During Execution

- The CPU first checks the cache for the required instruction or data before accessing RAM.
- If the data is available in the cache, it is processed immediately; otherwise, it is fetched from RAM.
- After processing by the ALU, the result is stored in registers, memory, or transferred to an I/O device.

7. Control Signal Flow

- The Control Unit generates control signals to manage communication between system components.
- These signals include **Memory Read**, **Memory Write**, **I/O Read**, **I/O Write**, and **Interrupt Handling**.
- The Control Bus carries these signals to synchronize all hardware operations during program execution.

8. Bottlenecks and Delays

- The concept map identifies performance bottlenecks such as CPU speed exceeding RAM speed, bus contention, and slow I/O devices.
- Heavy traffic on a single communication bus can delay data transfer between components.
- These bottlenecks reduce overall system efficiency and increase execution time.

9. Performance Improvements

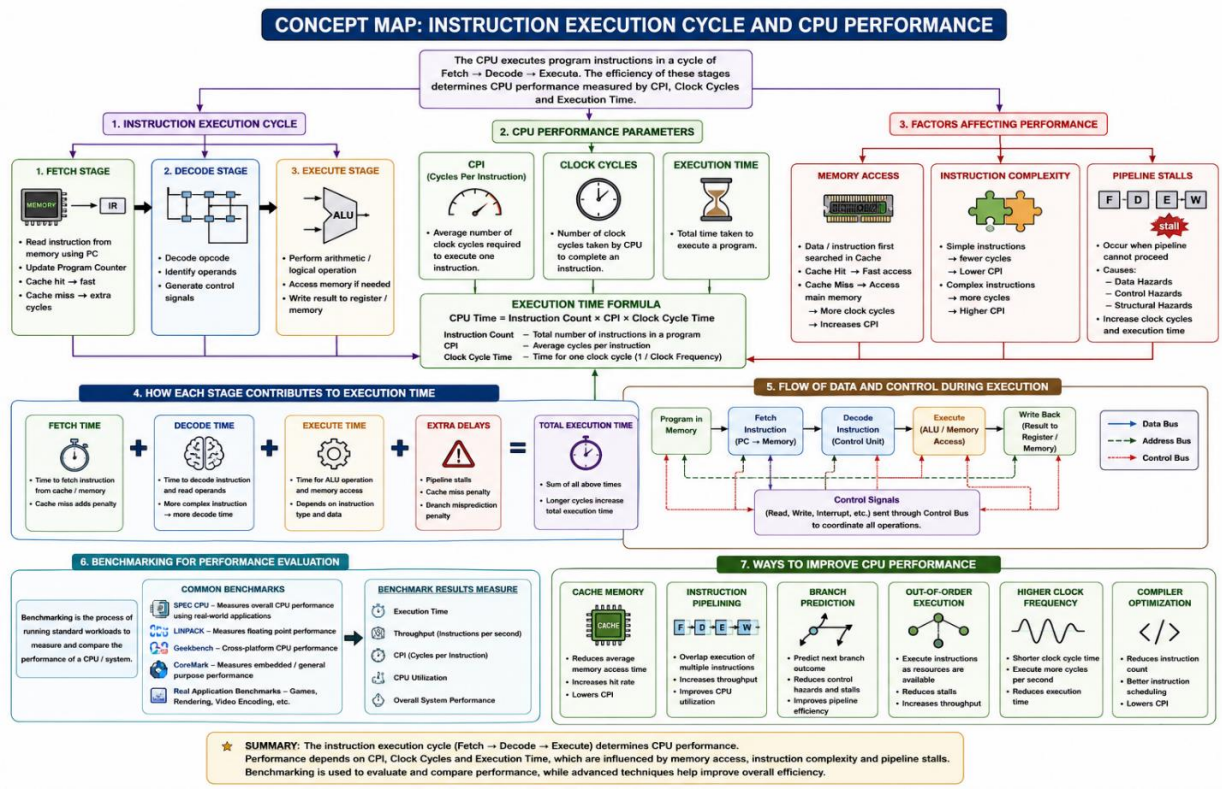
- Cache Memory reduces memory access time by storing frequently used data close to the CPU.
- Multi-Bus and Hierarchical Bus architectures improve communication speed by reducing bus contention and supporting parallel data transfer.

- Additional techniques such as **Direct Memory Access (DMA)**, **Registers**, and **Pipeline Execution** further enhance processing speed and system performance.

10. Overall Summary

- The concept map illustrates how the **CPU**, **Memory**, and **I/O Devices** interact through the Address, Data, and Control buses.
- It demonstrates the advantages of different bus architectures and explains the flow of data and control signals during instruction execution.
- Finally, it highlights common system bottlenecks and the hardware techniques used to improve computer performance, efficiency, and throughput.

2.Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.



1. Instruction Execution Cycle

- The concept map begins with the **Instruction Execution Cycle**, which consists of three stages: **Fetch**, **Decode**, and **Execute**.
- The **Fetch** stage retrieves the instruction from memory, the **Decode** stage interprets the instruction and generates control signals, and the **Execute** stage performs the required arithmetic, logical, or memory operation.
- These three stages work sequentially to complete the execution of every instruction and directly influence CPU performance.

2. CPU Performance Parameters

- The concept map links the execution cycle with three important performance metrics: **CPI (Cycles Per Instruction)**, **Clock Cycles**, and **Execution Time**.
- CPI represents the average number of clock cycles required to execute a single instruction, while Clock Cycles indicate the total processor cycles used during execution.
- Execution Time is determined using the formula **Execution Time = Instruction Count × CPI × Clock Cycle Time**, making it the primary measure of processor performance.

3. Factors Affecting Performance

- The image highlights three major factors that influence CPU performance: **Memory Access**, **Instruction Complexity**, and **Pipeline Stalls**.
- Memory access speed depends on cache hits and cache misses, instruction complexity determines the number of operations required, and pipeline stalls interrupt smooth instruction execution.
- Together, these factors increase or decrease the number of clock cycles required, thereby affecting the total execution time.

4. Contribution of Each Stage to Execution Time

- The concept map illustrates that the total execution time is the sum of **Fetch Time**, **Decode Time**, **Execute Time**, and additional delays.

- Delays caused by cache misses, branch mispredictions, and pipeline stalls increase the overall execution time of the processor.
- Efficient execution of each stage reduces CPU latency and improves processing speed.

5. Data and Control Flow During Execution

- The image shows how instructions flow from **Program Memory** through the **Fetch**, **Decode**, **Execute**, and **Write Back** stages.
- Data, address, and control buses enable communication between memory and the CPU during each stage.
- The **Control Unit** generates read, write, and interrupt signals to coordinate the complete execution process.

6. Benchmarking for Performance Evaluation

- The concept map includes benchmarking as a method to evaluate and compare CPU performance.
- Standard benchmarks such as **SPEC CPU**, **LINPACK**, **Geekbench**, and **CoreMark** are used to measure processor efficiency under different workloads.
- Benchmark results provide information about execution time, throughput, CPI, CPU utilization, and overall system performance.

7. Ways to Improve CPU Performance

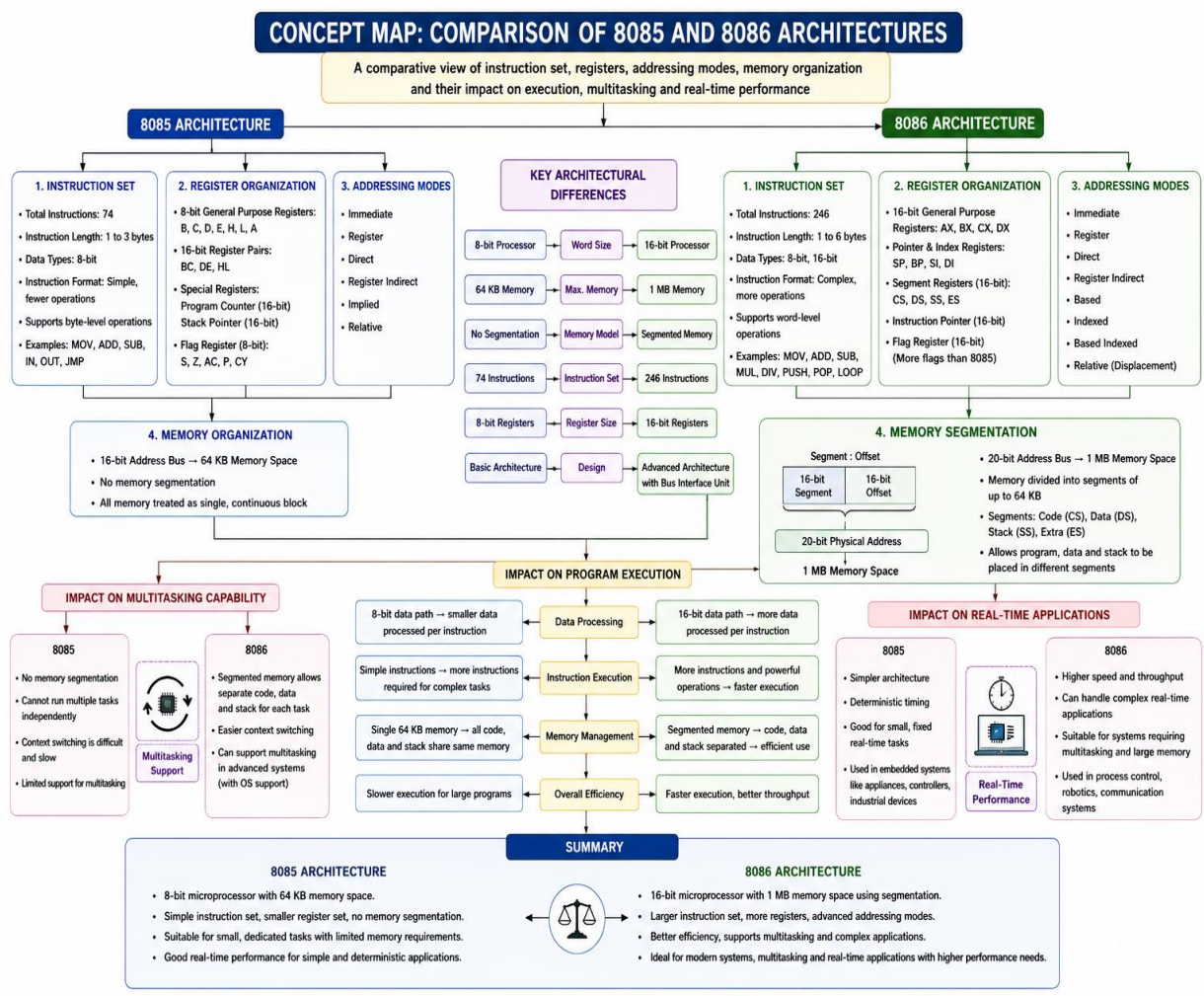
- The concept map presents several techniques used to improve processor performance, including **Cache Memory**, **Instruction Pipelining**, **Branch Prediction**, **Out-of-Order Execution**, **Higher Clock Frequency**, and **Compiler Optimization**.
- These techniques reduce memory access delays, minimize pipeline stalls, and increase instruction throughput.
- As a result, the CPU executes instructions more efficiently, reducing execution time and improving overall system performance.

8. Overall Summary

- The concept map demonstrates the relationship between the **Instruction Execution Cycle**, **CPU Performance Parameters**, and the factors that influence execution speed.

- It shows how memory access, instruction complexity, and pipeline stalls contribute to execution time and overall processor efficiency.
- Finally, it highlights the role of benchmarking and modern optimization techniques in evaluating and improving CPU performance.

3.Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.



1. 8085 Architecture

- The concept map presents the **8085** as an **8-bit microprocessor** with a simple architecture designed for basic computing applications.
- It supports a limited instruction set, 8-bit registers, and a maximum memory space of **64 KB** without memory segmentation.

- Its straightforward design makes it suitable for embedded systems and small real-time control applications.

2. 8086 Architecture

- The **8086** is shown as a **16-bit microprocessor** with a more advanced architecture than the 8085.
- It includes a larger instruction set, 16-bit registers, and a **1 MB memory space** using segmented memory organization.
- These architectural improvements provide higher processing speed, better efficiency, and support for more complex applications.

3. Instruction Set Comparison

- The concept map compares the instruction sets of both processors, showing that the **8085** supports around **74 instructions**, while the **8086** supports **246 instructions**.
- The 8085 mainly performs byte-level operations, whereas the 8086 supports both byte and word-level operations with more powerful instructions.
- A richer instruction set enables the 8086 to execute complex programs more efficiently than the 8085.

---

4. Register Organization

- The **8085** contains 8-bit general-purpose registers (A, B, C, D, E, H, and L) and register pairs for 16-bit operations.
- The **8086** contains 16-bit general-purpose registers (AX, BX, CX, DX), pointer registers, index registers, segment registers, and a flag register.
- The larger register organization of the 8086 reduces memory access and improves execution speed.

5. Addressing Modes

- The concept map shows that the **8085** supports basic addressing modes such as Immediate, Register, Direct, Register Indirect, and Implied.

- The **8086** provides additional addressing modes including Based, Indexed, Based Indexed, and Relative addressing.
- These advanced addressing modes make memory access more flexible and simplify complex program development.

6. Memory Organization and Segmentation

- The **8085** uses a single continuous **64 KB memory space** without segmentation.
- The **8086** introduces **memory segmentation**, dividing memory into Code, Data, Stack, and Extra segments to access up to **1 MB** of memory.
- Memory segmentation improves memory management, program organization, and efficient utilization of available memory.

7. Impact on Program Execution

- The concept map explains that the **8085** executes programs sequentially with an 8-bit data path, making execution suitable for simple applications.
- The **8086** uses a 16-bit data path and a Bus Interface Unit (BIU), enabling faster instruction fetching and execution.
- These features allow the 8086 to execute larger and more complex programs with higher efficiency.

8. Multitasking Capability

- The **8085** has limited multitasking capability because it lacks memory segmentation and advanced memory management features.
- The **8086** supports better multitasking by separating code, data, and stack into different memory segments.
- This architecture enables faster context switching and efficient execution of multiple tasks in operating system environments.

9. Real-Time Application Performance

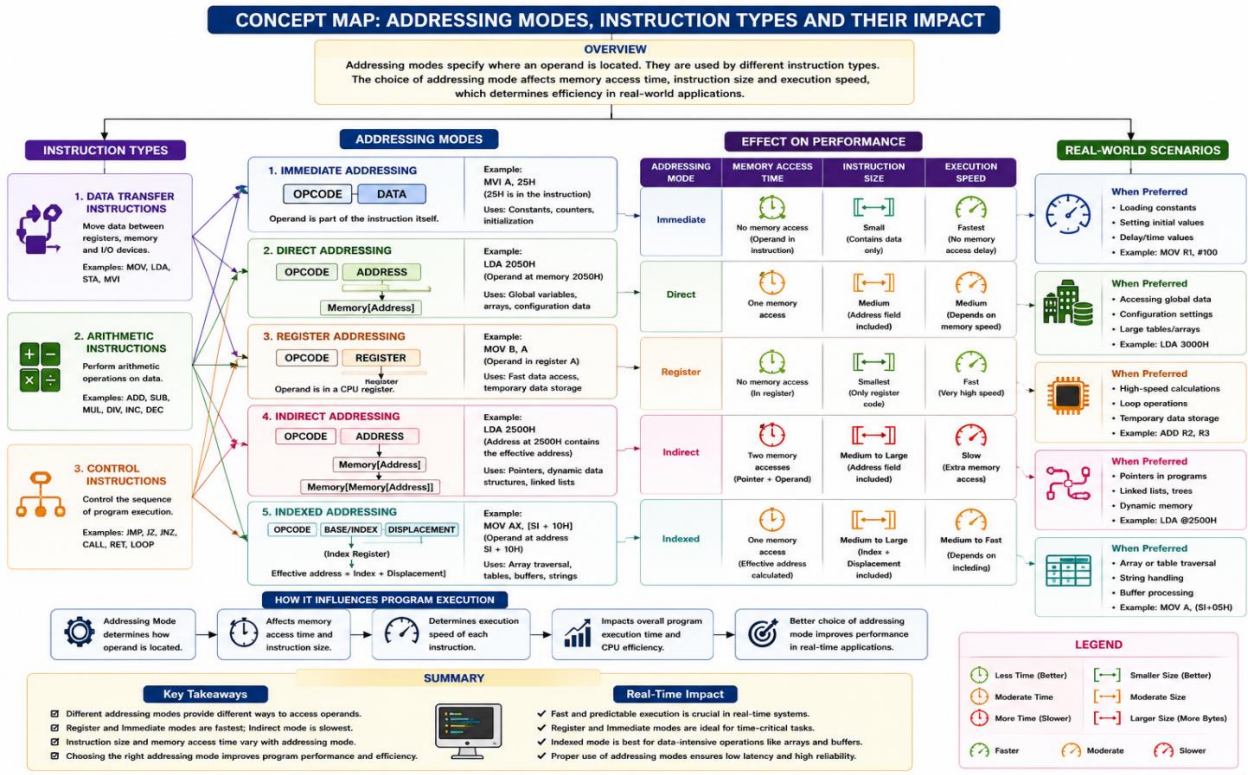
- The concept map compares the suitability of both processors for real-time applications.

- The **8085** is ideal for simple embedded systems requiring predictable timing and low hardware complexity.
- The **8086** is better suited for advanced real-time systems such as industrial automation, robotics, communication systems, and process control due to its higher speed and larger memory support.

10. Overall Summary

- The concept map highlights the architectural differences between the **8085** and **8086** in terms of instruction set, register organization, addressing modes, and memory management.
- It demonstrates how these features influence program execution speed, multitasking capability, and memory utilization.
- Overall, the **8085** is suitable for basic applications, whereas the **8086** provides greater performance, flexibility, and efficiency for complex and real-time computing systems.

4.Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.



1. Instruction Types

- The concept map begins by classifying instructions into **Data Transfer**, **Arithmetic**, and **Control Instructions**, which perform different operations during program execution.
- **Data Transfer Instructions** move data between registers, memory, and I/O devices, **Arithmetic Instructions** perform mathematical operations, and **Control Instructions** manage the execution flow of a program.
- Each instruction type is linked to one or more addressing modes depending on how operands are accessed.

2. Addressing Modes

- The concept map illustrates five addressing modes: **Immediate**, **Direct**, **Register**, **Indirect**, and **Indexed** addressing.
- Each addressing mode specifies a different method of locating the operand required by an instruction.
- The choice of addressing mode affects memory access, instruction size, execution speed, and overall processor efficiency.

3. Immediate Addressing Mode

- In **Immediate Addressing**, the operand is included directly within the instruction itself, eliminating the need for additional memory access.
- This addressing mode provides the **fastest execution speed** because the CPU can use the operand immediately after fetching the instruction.
- It is commonly used for **loading constants, initializing variables, counters, and delay values**.

4. Direct Addressing Mode

- In **Direct Addressing**, the instruction contains the memory address of the operand.
- The CPU performs **one memory access** to retrieve the required data, resulting in moderate execution speed.
- This mode is preferred for accessing **global variables, configuration data, and fixed memory locations**.

5. Register Addressing Mode

- In **Register Addressing**, the operand is stored inside a CPU register rather than in memory.
- Since no external memory access is required, this mode provides **very fast execution** and the smallest instruction size.
- It is widely used for **high-speed calculations, loop operations, and temporary data storage**.

6. Indirect Addressing Mode

- In **Indirect Addressing**, the instruction specifies a register or memory location that contains the effective address of the operand.
- Multiple memory accesses may be required, making this mode slower than immediate or register addressing.
- It is suitable for **pointer operations, linked lists, dynamic memory allocation, and data structures**.

7. Indexed Addressing Mode

- In **Indexed Addressing**, the effective memory address is calculated by adding an index register to a displacement value.
- This addressing mode is ideal for accessing sequential data stored in arrays, tables, and strings.
- It provides efficient data processing for applications involving **array traversal, buffer management, and string manipulation**.

8. Effect on Performance

- The concept map compares the impact of each addressing mode on **memory access time, instruction size, and execution speed**.
- **Immediate** and **Register Addressing** require little or no memory access and therefore provide the fastest execution, while **Indirect Addressing** is slower because of additional memory references.
- Selecting the appropriate addressing mode improves CPU efficiency and reduces overall program execution time.

9. Real-World Scenarios

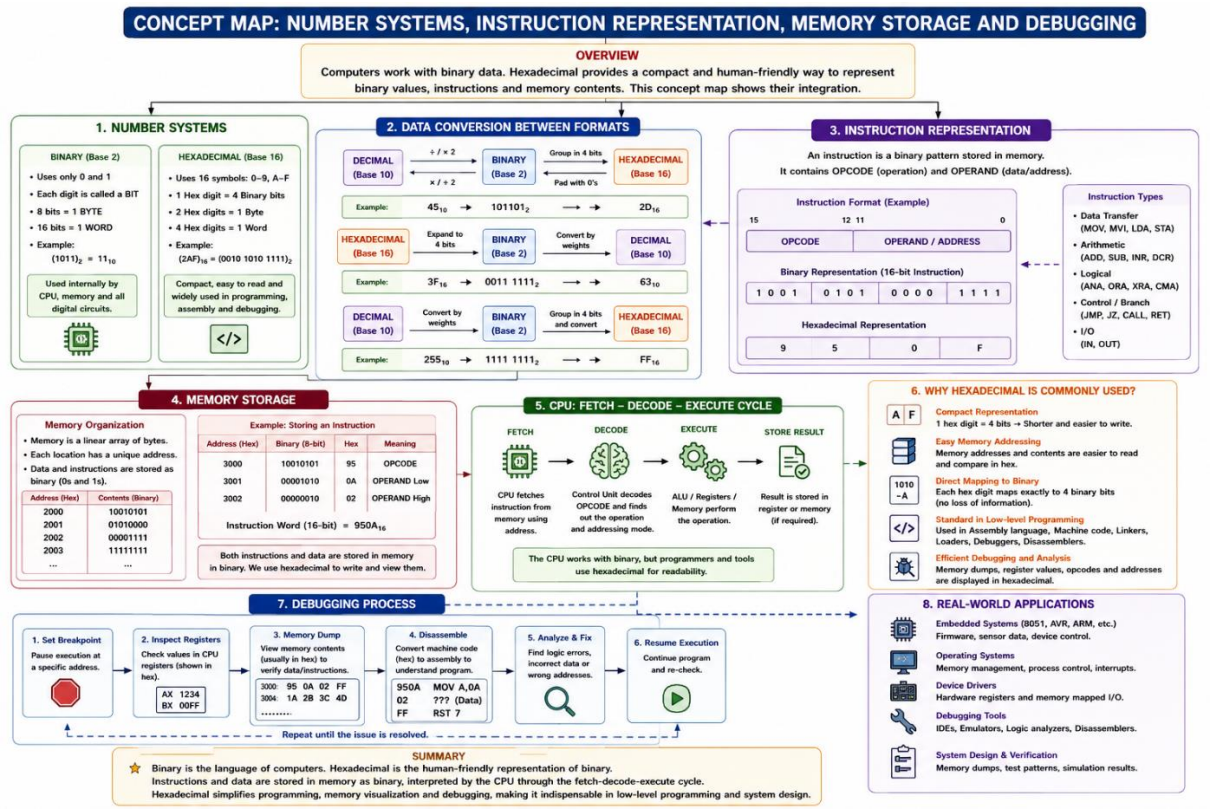
- The concept map illustrates practical situations where each addressing mode is preferred in real-world applications.
- Immediate addressing is used for constants, Register addressing for high-speed computation, Direct addressing for fixed data, Indirect addressing for pointers, and Indexed addressing for arrays and tables.

- Choosing the correct addressing mode improves program performance, memory utilization, and execution efficiency in embedded systems and general-purpose computing.

10. Overall Summary

- The concept map demonstrates the relationship between **addressing modes**, **instruction types**, and **processor performance**.
- It shows how each addressing mode influences memory access time, instruction size, execution speed, and the selection of suitable instruction types.
- Overall, the map emphasizes that selecting the appropriate addressing mode is essential for developing efficient, fast, and optimized computer programs in both real-time and general computing applications.

5.Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.



1. Number Systems

- The concept map begins with the two fundamental number systems used in computing: **Binary (Base-2)** and **Hexadecimal (Base-16)**.
- Binary uses only the digits **0** and **1**, which are directly understood by digital circuits, while hexadecimal uses the symbols **0–9** and **A–F** for compact representation.
- These number systems form the basis for data representation, instruction encoding, and memory organization in computer systems.

2. Data Conversion Between Formats

- The concept map illustrates the conversion of data between **Decimal**, **Binary**, and **Hexadecimal** formats.
- Binary values are grouped into sets of four bits (nibbles), allowing direct conversion to hexadecimal without loss of information.
- Data conversion enables programmers and computer systems to represent the same value in different formats for processing and analysis.

3. Instruction Representation

- The concept map explains that every machine instruction consists of an **Opcode** (operation code) and an **Operand** (data or address).
- Instructions are stored internally in **binary format**, while programmers often view and write them in hexadecimal form for better readability.
- Different instruction types, such as **Data Transfer**, **Arithmetic**, **Logical**, **Control**, and **I/O Instructions**, are represented using binary bit patterns.

4. Memory Storage

- The memory section shows that both **instructions and data** are stored in memory as binary values.
- Every memory location has a unique address, which is commonly represented in hexadecimal for easier identification.
- The concept map demonstrates how binary data is physically stored, while hexadecimal simplifies memory visualization and programming.

5. CPU Fetch–Decode–Execute Cycle

- The concept map illustrates how the CPU processes instructions through the **Fetch**, **Decode**, and **Execute** stages.
- During the **Fetch** stage, the instruction is retrieved from memory, the **Decode** stage interprets the opcode and addressing mode, and the **Execute** stage performs the required operation using the ALU and registers.
- This cycle enables the CPU to process binary instructions and produce the required output.

6. Why Hexadecimal is Commonly Used

- The concept map explains that hexadecimal provides a **compact and human-readable representation** of binary data.
- Since **one hexadecimal digit represents exactly four binary bits**, it simplifies the reading and writing of memory addresses, instructions, and machine code.
- Hexadecimal is widely used in **assembly programming, system design, firmware development, debugging, and memory analysis**.

7. Debugging Process

- The debugging section illustrates the steps involved in identifying and correcting software or hardware errors.
- It includes activities such as setting **breakpoints**, inspecting **register values**, viewing **memory dumps**, disassembling machine code, analyzing errors, and resuming execution.
- Most debugging tools display addresses and machine code in hexadecimal because it is shorter and easier to interpret than binary.

8. Real-World Applications

- The concept map highlights practical applications of binary and hexadecimal in modern computing systems.

- These include **embedded systems**, **8085/8086 microprocessor programming**, **operating systems**, **device drivers**, **firmware development**, and **debugging tools**.
- Using hexadecimal improves programming efficiency, simplifies memory management, and supports reliable low-level system development.

9. Relationship Between Components

- The concept map connects **number systems**, **instruction representation**, **memory storage**, **CPU processing**, and **debugging** into a continuous workflow.
- Binary data is converted into hexadecimal for readability, stored in memory, interpreted by the CPU during execution, and analyzed during debugging.
- These relationships demonstrate how each component contributes to efficient program execution and system operation.

10. Overall Summary

- The concept map integrates **binary and hexadecimal number systems** with **instruction representation**, **memory organization**, and **CPU execution**.
- It explains how data is converted between formats, how instructions are stored and interpreted, and how hexadecimal simplifies low-level programming and debugging.
- Overall, the concept map demonstrates the importance of number systems in computer architecture, memory management, processor operation, and system design.